Our Method	DocLayout-YOLO
Document Title	Title
Paragraph Title	Title
Text	Plain Text
Page Number	Abandon
Abstract	Plain Text
Content	Plain Text
Reference	Plain Text
Footnote	Abandon
Header	Abandon
Footer	Abandon
Algorithm	Plain Text
Formula	Isolate Formula
Formula Number	Formula Number
Image	Figure
Image Caption	Figure Caption
Table	Table
Table Caption	Table Caption
Chart	Figure
Chart Caption	Figure Caption
Seal	Figure
Header Image	Abandon
Footer Image	Abandon
Aside Text	Abandon

Table 1. Comparison of categories recognized by our method and another advanced algorithm DocLayout-YOLO [14].

along with the real labeled data, form a comprehensive training set that enhances the learning process of the student models PP-DocLayout-S and PP-DocLayout-M. By integrating the high-quality pseudo-labels with the labeled data, the models can generalize better, learning robust features that improve document layout detection.

4. Experimental Results

4.1. Dataset

we collected a comprehensive dataset of document images encompassing a wide variety of types such as Chinese and English academic papers, magazines, newspapers, research reports, exam papers, handwritten notes, contracts, and books. This diverse dataset ensures the robustness and generalizability of our model across different document formats and structures. The dataset comprises 30,000 images for training and 500 images for evaluation. Images are collected from Baidu image search and public datasets, including Doclaynet [6] and PublayNet [15]. Images are annotated with 23 common layout categories and the distribution of these categories is detailed in the Table 5 in the appendix.

As shown in **Table 1**, our method defines a more comprehensive and fine-grained set of categories compared to DocLayout-YOLO [14]. While DocLayout-YOLO simplifies many document elements into broad classes such as "title," "text," and "figure," our method distinguishes between semantically meaningful elements like document titles, paragraph titles, page numbers, headers, footers, and footnotes. This granularity enables better parsing of the document's hierarchical structure and logical relationships. Additionally, our method accurately identifies and categorizes high-value elements such as formulas, charts, and seals, which DocLayout-YOLO either misclassifies or ignores (e.g., labeling them as "abandon" or "figure"). This comprehensive categorization supports a wider range of downstream tasks, including document understanding, information extraction, and format conversion.

4.2. Implementation Details

The **PP-DocLayout-L** model is built upon the RT-DETR-L [13] object detection architecture and utilizes a pre-trained PPHGNetV2-B4 model that has undergone knowledge distillation. The training was configured with a constant learning rate of 0.0001. The model was trained for 100 epochs with a batch size of 2 per GPU, using a total of 8 GPUs, resulting in a total training time of approximately 26 hours on NVIDIA V100 GPUs. The **PP-DocLayout-M** and **PP-DocLayout-S** models are based on the PicoDet-M and PicoDet-S [12] object detection architecture, respectively. Both models were trained for 100 epochs with a batch size of 2 per GPU, utilizing a total of 8 GPUs. The learning rates were set to for 0.02 PP-DocLayout-M and 0.06 for PP-DocLayout-S, and were dynamically adjusted using the CosineDecay [5] learning rate scheduler.

4.3. Main Results

Table 2 presents the performance and specifications of different variants of the PP-DocLayout model. The table highlights the trade-offs between accuracy, inference speed, and model size for each model variant.

The PP-DocLayout-L model achieves the highest accuracy with a mean Average Precision (mAP) of 90.4% at an IoU threshold of 0.5. However, this accuracy comes with a model size of 30.94 million parameters and inference times, taking 13.39 milliseconds on a T4 GPU, about 74.6 FPS and approximately 759.76 milliseconds on a CPU, which translates to about 1.32 FPS. Referring to **Figure 4** in the appendix, we provide additional visualization results to further demonstrate the effectiveness of our model across a diverse range of document types and layouts. Specifically, we visualize the performance of our method on documents such as *papers*, *magazines*, *newspapers*, *research reports*, *books*, *notebooks*, *contracts* and *test papers*. The visualizations clearly show that our model accurately identifies and categorizes diverse elements.

The PP-DocLayout-S model offers a significantly smaller model size of 1.21 million parameters with faster inference times 8.11 milliseconds on a T4 GPU, about 123 FPS and 14.49 milliseconds on a CPU, which translates to

Model Name	mAP@0.5 (%)	GPU Inference (ms)	CPU Inference (ms)	Params (M)
PP-DocLayout-L	90.4	13.39	759.76	30.94
PP-DocLayout-M	75.2	12.73	59.82	5.65
PP-DocLayout-S	70.9	8.11	14.49	1.21

Table 2. Model performance and specifications. *Note:* GPU inference times are based on an NVIDIA Tesla T4 machine. CPU inference speeds are based on an Intel(R) Xeon(R) Gold 6271C CPU @ 2.60GHz, with 8 threads and FP16 precision.

approximately 69.04 FPS. Despite its compactness, it maintains a respectable mAP of 70.9%.

The PP-DocLayout-M model strikes a balance between the two extremes, achieving an mAP of 75.2%. It has a moderate model size of 5.65 million parameters, with inference times of 12.73 milliseconds on a T4 GPU and about 59.82 milliseconds on a CPU, translating to approximately 16.72 FPS.

These results illustrate the trade-offs inherent in model design, where increases in accuracy often come at the expense of model size and inference speed. The choice of model may thus depend on the specific requirements of accuracy, computational resources, and latency in practical applications.

4.4. Qualitative Analysis

In this section, we compare the results of our proposed method with the advanced method DocLayout-YOLO [14] through visualization. Due to differences in label categories, traditional quantitative metrics cannot be directly applied. Therefore, we employ visualization techniques to present the performance of each method enabling an intuitive comparison.

The visualization results are shown in **Figure 3**. The first row presents the results of DocLayout-YOLO [14], while the second row shows the results of our method. From the first column of images, it can be observed that our results include elements such as document titles, abstracts, paragraph titles and text, which are essential for understanding the semantic hierarchy and logical structure of the document. In contrast, DocLayout-YOLO categorizes these elements into only two broad classes, "title" and "plain text," which limits its ability to parse the document's semantic layers effectively. Additionally, our PP-DocLayout can accurately locate page numbers, headers, footers, and footnotes, whereas DocLayout-YOLO often classifies these as "abandon," disregarding their potential value.

The second column highlights the difference in formula recognition. Our method is capable of identifying both inline and block-level formulas, which is crucial for downstream tasks such as PDF-to-Markdown conversion. In contrast, DocLayout-YOLO struggles to recognize inline formulas, focusing only on prominent block-level formulas, which hinders its utility in tasks requiring full-text recognition.

In the third column, the comparison demonstrates our method's superior performance in handling handwritten notes. While our approach correctly identifies and categorizes handwritten content, DocLayout-YOLO misclassifies it as "figure" failing to capture its textual significance.

Finally, the fourth column illustrates our method's ability to distinguish between natural images, charts, and seals. Charts and seals are particularly important as they often contain high-value information, and our method ensures they are categorized separately. DocLayout-YOLO, on the other hand, does not make this distinction, potentially overlooking critical details.

Overall, our method provides a more granular and accurate representation of document elements, enabling better semantic understanding and supporting a wider range of downstream tasks compared to DocLayout-YOLO.

4.5. Ablation Study

In order to evaluate the impact of semi-supervised learning and knowledge distillation on model performance, we conducted a series of ablation experiments using the PP-DocLayout model variants. We compared the performance of each model with and without these techniques, measuring the mean Average Precision (mAP) at an IoU threshold of 0.5.

Knowledge Distillation We examined the effect of knowledge distillation on the PP-DocLayout-L variant as shown in **Table 3**, The use of knowledge distillation yielded an mAP improvement from 89.3% to 90.4%, demonstrating a positive impact on model accuracy.

Algorithm Name	Knowledge Distillation	mAP@0.5 (%)
PP-DocLayout-L	No	89.3
PP-DocLayout-L	Yes	90.4 (+1.1)

Table 3. Results of the ablation study on the effect of knowledge distillation.

Semi-supervised Learning As shown in **Table 4**, for the PP-DocLayout-M and PP-DocLayout-S models, incorporating semi-supervised learning significantly enhanced performance. Specifically, the mAP for PP-DocLayout-M improved from 73.8% to 75.2%, reflecting a 1.4% increase.

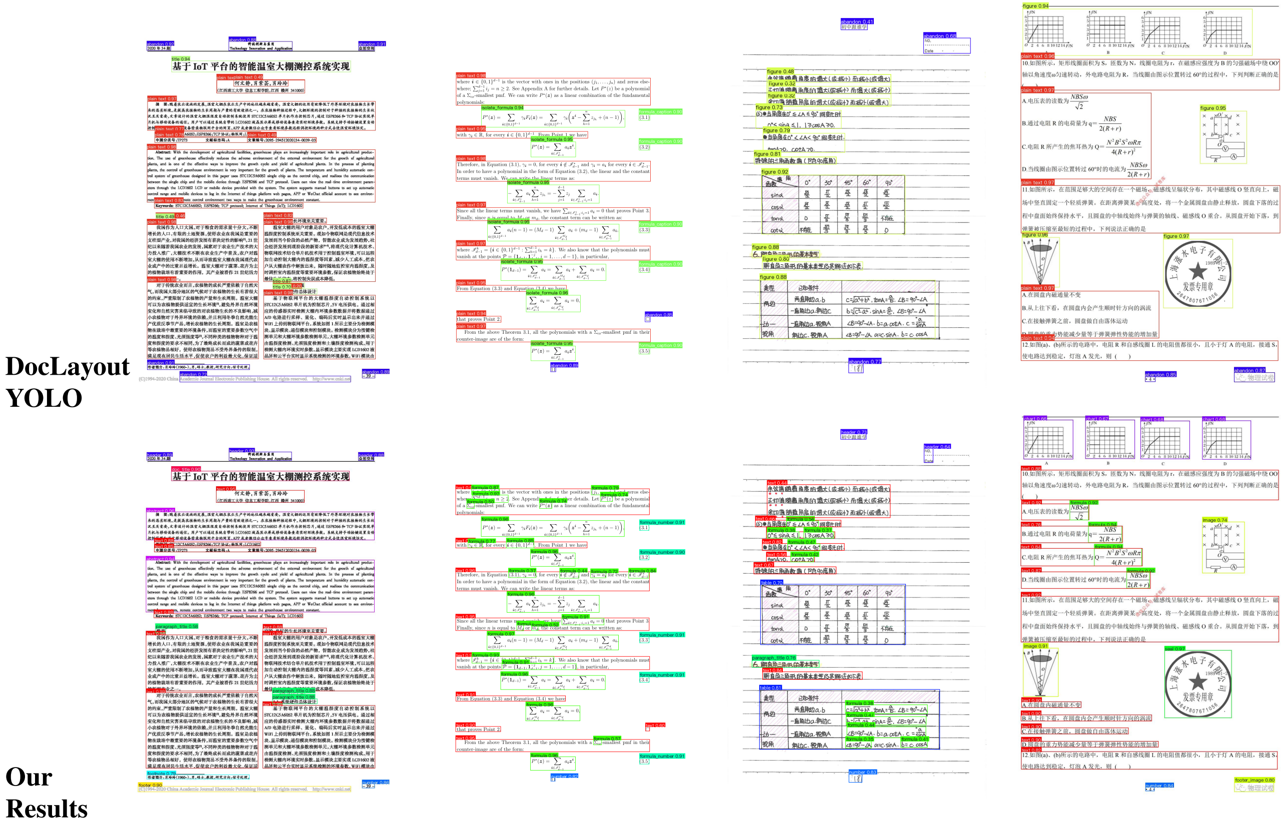

Figure 3. Comparison of our results and DocLayout-YOLO [14].

Similarly, PP-DocLayout-S saw an increase from 66.2% to 70.9%, marking a substantial improvement of 4.7%.

Algorithm Name	Semi-Supervised Learning	mAP@0.5 (%)
PP-DocLayout-M	No	73.8
PP-DocLayout-M	Yes	75.2 (+1.4)
PP-DocLayout-S	No	66.2
PP-DocLayout-S	Yes	70.9 (+3.7)

Table 4. Ablation study results on the effect of semi-supervised learning.

These results underscore the efficacy of both semi-supervised learning and knowledge distillation in enhancing model performance, thereby supporting their application in document layout analysis tasks.

5. Conclusion

We have introduced PP-DocLayout, a novel document layout detection model developed within the PaddlePaddle framework, to address the significant challenges faced by existing layout detection models in document intelligence. PP-DocLayout represents a significant step forward in document layout analysis, offering a versatile and efficient solution to address the complexities and diversities of document structures. Our models not only push the boundaries of the state of the art but also provide practical tools for real-world applications, paving the way for future advancements in document intelligence and related fields.

References

[1] Cheng Da, Chuwei Luo, Qi Zheng, and Cong Yao. Vision grid transformer for document layout analysis. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 19462–19472, 2023. 2

[2] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. 3

[3] Yupan Huang, Tengchao Lv, Lei Cui, Yutong Lu, and Furu Wei. Layoutlmv3: Pre-training for document ai with unified text and image masking. In *Proceedings of the 30th ACM International Conference on Multimedia*, pages 4083–4091, 2022. 2

[4] Junlong Li, Yiheng Xu, Tengchao Lv, Lei Cui, Cha Zhang, and Furu Wei. Dit: Self-supervised pre-training for document image transformer. In *Proceedings of the 30th ACM International Conference on Multimedia*, pages 3530–3539, 2022. 2

[5] Ilya Loshchilov and Frank Hutter. Sgdr: Stochas-